

Augmented reality wireframe cube

Contents

1	Preliminaries	1
1.1	Outline of the exercise	1
1.2	Description of the input data	1
1.3	Notations and coordinate systems	2
2	Part 1: Drawing a cube on the undistorted images	3
2.1	Reminder: Perspective projection	3
2.1.1	Equation of perspective projection	3
2.1.2	Axis-angle representation for rotations	4
2.2	Writing and testing the projection function	4
2.3	Drawing the cube	4
2.4	(Optional) Generating a video from the images	4
3	Part 2: Accounting for lens distortion	5
3.1	Lens distortion modelling	5
3.2	Writing and testing the projection function with lens distortion	5
3.3	Undistorting the images	6
4	Numerical Exercises	8

1 Preliminaries

1.1 Outline of the exercise

The goal of this exercise is to superimpose a virtual cube on a video of a planar grid viewed from different orientations. **In this exercise, the 3D positions of the checkerboard, and the relative camera poses are provided**, as well as the intrinsics of the camera. The purpose of this exercise is to familiarize with the basics of perspective projection, change of coordinate systems and lens distortion, as well as basic image processing with Matlab/Python.

In the first part of the exercise, you will be given images that have already been compensated for distortion, and you will write a function that draws a virtual cube on the compensated image.

In the second part, you will implement a simple distortion and use it to undistort the camera image.

Note The provided `requirements.txt` can be used to install the Python dependencies and assumes you are using Python 3.12.3.

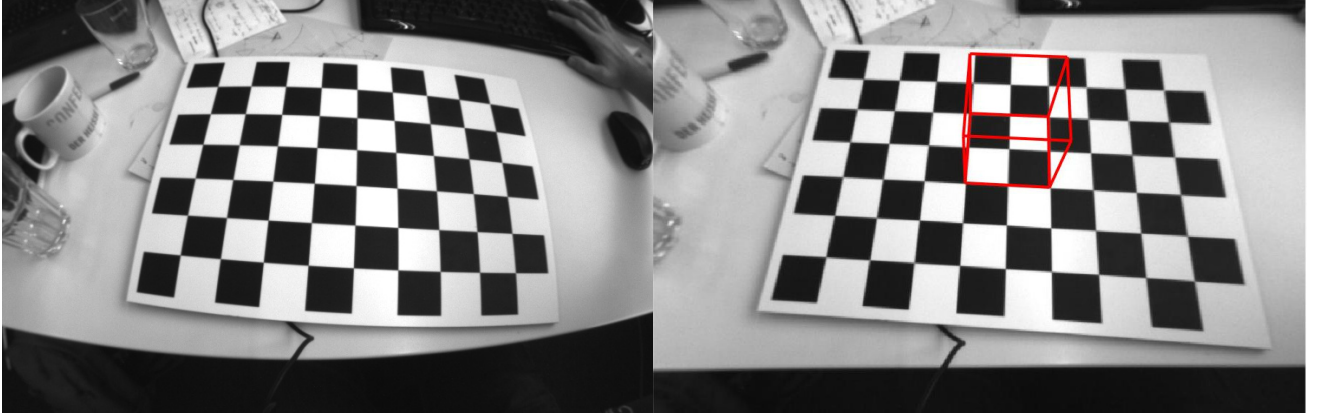


Figure 1: Input image (left) and expected output (right): undistorted image with a virtual cube superimposed

1.2 Description of the input data

The **data/** folder contains the inputs that you will need to complete these exercises. *You should be able to load all text files with the `load` command of Matlab or `np.loadtxt` in Python.*

- **images/** contains a sequence of images recorded by a camera moving around a checkerboard pattern.
- **images_undistorted/** contains images that have been processed to compensate for lens distortion. You will use them in the first part of the exercise. In the second part, you will write code to generate these compensated images from the original images yourself.
- **K.txt** and **D.txt** contain the intrinsics of the camera
- **poses.txt** contains, for each image, the world-to-camera transformation $\mathbf{T}_{CW}$ that maps points from the world frame W (defined below) into the camera frame C :

$${}_C\tilde{\mathbf{P}} = \mathbf{T}_{CW} {}_W\tilde{\mathbf{P}}.$$

This is *not* the pose of the camera with respect to the world (that would be $\mathbf{T}_{WC} = \mathbf{T}_{CW}^{-1}$). Line i stores $\mathbf{T}_{CW}^{(i)}$ as a 6-tuple $(\omega_x, \omega_y, \omega_z, t_x, t_y, t_z)$, where $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$ is the axis-angle representation (section 2.1.2) of the rotation $\mathbf{R}_{CW}$, and ${}_C\mathbf{t}_{CW} = (t_x, t_y, t_z)^\top$ is the origin of W expressed in C , in meters.

1.3 Notations and coordinate systems

In this exercise, we use the following conventions. The left subscript is the frame in which a quantity is expressed. We write $\mathbf{R}$ for rotations (not $\mathbf{C}$).

- ${}_A\mathbf{P}$ denotes that the Euclidean point $\mathbf{P}$ is expressed in frame A . The homogeneous counterpart is ${}_A\tilde{\mathbf{P}}$.
- $\mathbf{R}_{BA}$ maps vectors from A to B (orientation of A with respect to B):

$${}_B\mathbf{v} = \mathbf{R}_{BA} {}_A\mathbf{v}.$$

- $\mathbf{T}_{BA}$ maps points from A to B (pose of A with respect to B):

$${}_B\tilde{\mathbf{P}} = \mathbf{T}_{BA} {}_A\tilde{\mathbf{P}}, \quad \mathbf{T}_{BA} = \begin{bmatrix} \mathbf{R}_{BA} & {}_B\mathbf{t}_{BA} \\ \mathbf{0} & 1 \end{bmatrix},$$

where ${}_B\mathbf{t}_{BA}$ is the origin of A expressed in B .

The reference (or world) frame, denoted W , is right-handed and centered on the upper left corner of the checkerboard, as illustrated in Figure 2. The size of each square of the checkerboard is 4 cm. The camera frame is denoted C .

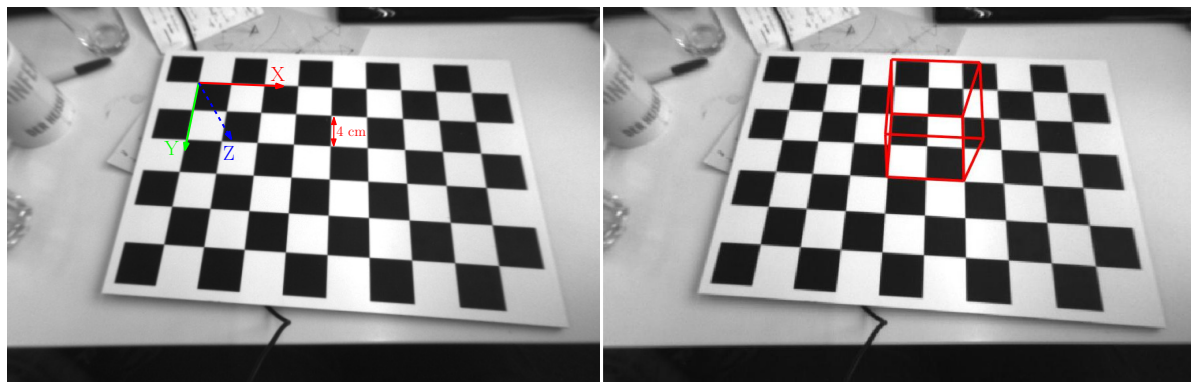


Figure 2: World frame W (left), and superimposed cube lying on the checkerboard (right).

2 Part 1: Drawing a cube on the undistorted images

In this section, you will work with an image that has been already compensated for lens distortion. Your goal will be to create a 3D cube lying on the checkerboard and project it into the image (Figure 2).

You will first write a function that projects world points to a given image (using the corresponding world-to-camera transformation), and test it by reprojecting the checkerboard corners on the image. Once your projection function works properly, you will create a cube in the world frame and draw it on the image.

2.1 Reminder: Perspective projection

Figure 3 is a reminder of the different steps involved in projecting a 3D point ${}_W\mathbf{P}$ (expressed in the world frame) to the image plane of camera C , when the camera matrix K and the extrinsics $\mathbf{R}_{CW}$ and ${}_C\mathbf{t}_{CW}$ are known.

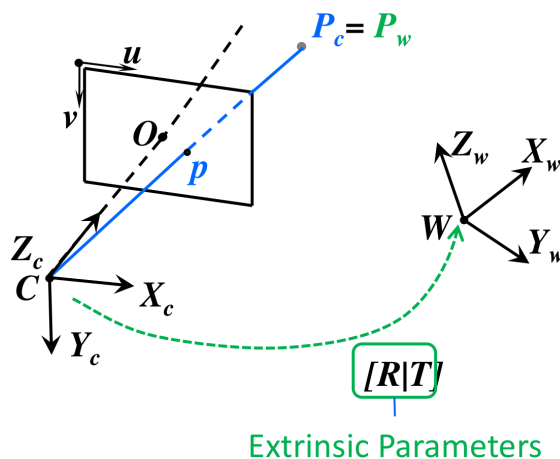


Figure 3: Perspective projection: ${}_W\mathbf{P}$ is first expressed in the camera frame as ${}_C\mathbf{P} = \mathbf{R}_{CW} {}_W\mathbf{P} + {}_C\mathbf{t}_{CW}$, then mapped to pixel coordinates $(u, v)^\top$ by K . (The figure labels a single physical point as $P_c = P_w$ and writes $[R|T]$ for the extrinsics; those coordinates are not equal, and $[R|T]$ stands for $[\mathbf{R}_{CW} | {}_C\mathbf{t}_{CW}]$.)

2.1.1 Equation of perspective projection

Assuming the lens distortion has already been compensated (which is the case in this section), the perspective projection can be written *linearly* in homogeneous coordinates as shown in the lecture:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[\mathbf{R}_{CW} | \mathbf{c}\mathbf{t}_{CW}] {}_w\tilde{\mathbf{P}} \quad (1)$$

where $(u, v)^\top$ is the projection in pixel coordinates and

$${}_w\mathbf{P} = \begin{bmatrix} X_W \\ Y_W \\ Z_W \end{bmatrix}, \quad {}_w\tilde{\mathbf{P}} = \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}.$$

K is the 3×3 camera matrix, provided in ***K.txt***. The 3×4 block $[\mathbf{R}_{CW} | \mathbf{c}\mathbf{t}_{CW}]$ is the upper part of $\mathbf{T}_{CW}$.

2.1.2 Axis-angle representation for rotations

In this exercise, the rotation $\mathbf{R}_{CW}$ (from the world frame to the camera frame) is given using the axis-angle representation. A 3D rotation is parameterized by a 3D vector $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)^\top$, where $\hat{\boldsymbol{\omega}} = \boldsymbol{\omega}/\|\boldsymbol{\omega}\|$ is the unit axis of rotation and $\|\boldsymbol{\omega}\| = \theta$ is the rotation angle. Rodrigues' rotation formula converts this representation to a rotation matrix:

$$\mathbf{R}_{CW} = I + (\sin \theta)[\hat{\boldsymbol{\omega}}]_\times + (1 - \cos \theta)[\hat{\boldsymbol{\omega}}]_\times^2$$

where

$$[\hat{\boldsymbol{\omega}}]_\times = \begin{bmatrix} 0 & -\hat{\omega}_z & \hat{\omega}_y \\ \hat{\omega}_z & 0 & -\hat{\omega}_x \\ -\hat{\omega}_y & \hat{\omega}_x & 0 \end{bmatrix}$$

is the *cross-product matrix* of $\hat{\boldsymbol{\omega}}$.

2.2 Writing and testing the projection function

We will work with the first image, located in `/data/images_undistorted/img_0001.jpg`.

- (**main.py**) Read the image
 - into Matlab (*imread*)
 - into Python (*cv2.imread*)
 and convert it to grayscale (rgb2gray in MATLAB or cv2.IMREAD_GRAYSCALE in Python).
- (**main.py**) Create a matrix containing the 3D positions of all the checkerboard corners in the world frame ${}_w\mathbf{P}$. You may find
 - *meshgrid* in MATLAB
 - *np.meshgrid* in Python
 helpful to achieve this.
- (**project_points.py** and **pose_vector_to_transformation_matrix.py**) Write a function to project the corners ${}_w\mathbf{P}$ on the image plane. You will need $[\mathbf{R}_{CW} | \mathbf{c}\mathbf{t}_{CW}]$ (the upper 3×4 block of $\mathbf{T}_{CW}$), which you can read from the first line of ***poses.txt*** as $(\omega_x, \omega_y, \omega_z, t_x, t_y, t_z) = (\boldsymbol{\omega}, \mathbf{c}\mathbf{t}_{CW})$. You will find it convenient to write two functions *poseVectorToTransformationMatrix* and *projectPoints*.
- (**main.py**) Superimpose the projected corners to the undistorted image using

- `scatter` in MATLAB
- matplotlib function `pyplot.plot` in Python

for example. The output should look like Figure 4 if your code works properly.

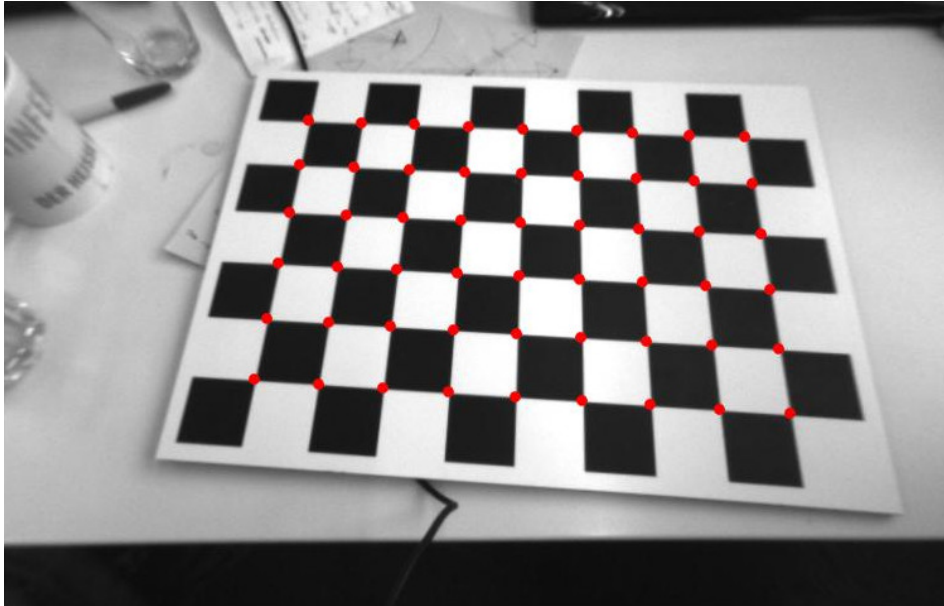


Figure 4: Expected output: the checkerboard corners are reprojected (in red) at their correct position on the undistorted image.

2.3 Drawing the cube

- Write some code to create a matrix containing the 8 vertices of a cube lying on the checkerboard's plane. The position of the cube on the checkerboard and its size should be customizable.
- Project the cube's vertices on the image and draw a line (*line*) for each edge of the cube.

Figure 1 illustrates the expected output.

2.4 (Optional) Generating a video from the images

Repeat the process above for all the images in the sequence and generate a small movie (at 30 frames per second). <https://ch.mathworks.com/help/matlab/examples/convert-between-image-sequences-and-video.html>

3 Part 2: Accounting for lens distortion

3.1 Lens distortion modelling

Real camera lenses are not ideal and introduce some distortion in the image. To account for these non-idealities, it is necessary to add a distortion model to the equations of perspective projection. A simple *radial distortion model* was introduced during the lecture. In this exercise, we use this model, and simply add a higher-order term, parameterized by an additional variable k_2 . The distortion model is therefore fully parameterized by two variables (k_1, k_2) that are provided in the file ***D.txt***.

Because the distortion model is not linear, the projection function needs to split into two steps.

- We start again by mapping the world point ${}_w\mathbf{P}$ to undistorted pixel coordinates $\mathbf{p} = (u, v)^\top$, as in Equation 1:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[\mathbf{R}_{CW} | \mathbf{c}_{CW}]_w \tilde{\mathbf{P}}$$

- Next, we apply lens distortion to $\mathbf{p}$ to get the *distorted pixel coordinates* $\mathbf{p}_d = (u_d, v_d)^\top$:

$$\begin{pmatrix} u_d \\ v_d \end{pmatrix} = (1 + k_1 r^2 + k_2 r^4) \begin{pmatrix} u - u_0 \\ v - v_0 \end{pmatrix} + \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} \quad (2)$$

where $r^2 = (u - u_0)^2 + (v - v_0)^2$ is the squared radial distance of $\mathbf{p}$ from the principal point, and u_0, v_0 are the principal-point coordinates from K . The subscript d means *distorted*; it is not a coordinate frame.

3.2 Writing and testing the projection function with lens distortion

- (`main.py`) Read the image `/data/images/img_0001.jpg` which is, this time, not compensated for distortion.
- (`project_points.py` and `distort_points.py`) Modify your function `project_points` to take into account the lens distortion, as described above.
- (`main.py`) Project the checkerboard corners in the distorted image. The expected output is shown in Figure 5.



Figure 5: Expected output: the checkerboard corners are reprojected (in red) at their correct position on the non-corrected image.

3.3 Undistorting the images

We will now use the new projection function (that takes distortion into account) to generate an undistorted image from the original image. Let I_d and I_u be respectively the distorted and undistorted images.

A naive way to undistort I_d would be through forward warping, i.e. warp every pixel $(u_d, v_d)^\top$ in I_d to I_u as follows:

$$I_u(\Gamma^{-1}(u_d, v_d)) = I_d(u_d, v_d)$$

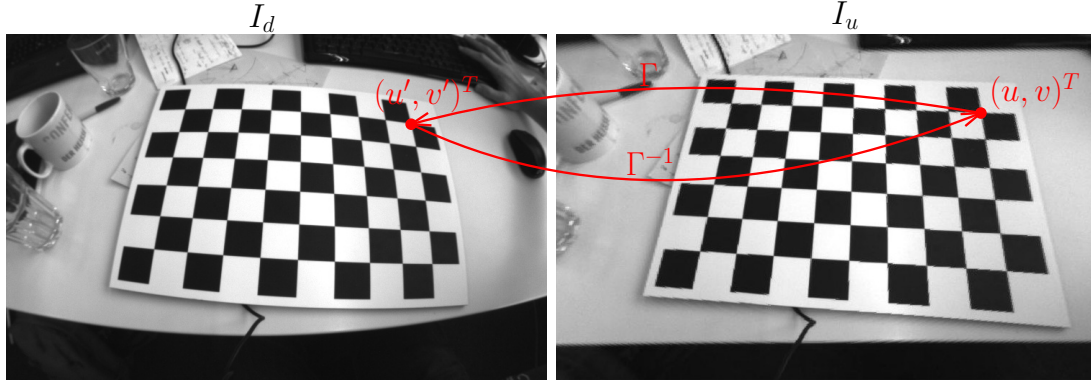


Figure 6: Distorted pixel coordinates $(u_d, v_d)^\top$ are obtained from undistorted coordinates $(u, v)^\top$ through the distortion function Γ .

where $\Gamma(u, v) = (u_d, v_d)$ is the distortion function that maps undistorted pixel coordinates $(u, v)^\top$ to distorted pixel coordinates $(u_d, v_d)^\top$.

However, due to the undistorted pixel locations being non-integer, the resulting image would have some artifacts. Moreover, inverting the distortion function Γ amounts to solving a system of polynomial equations, which is costly.

In image processing, this is commonly solved by doing backward warping, i.e. warping pixel locations from the *destination image* to the *source image*. In our case, we want to warp from the undistorted image to the distorted image:

$$I_u(u, v) = I_d(\Gamma(u, v)) \quad (3)$$

Since $\Gamma(u, v) = (u_d, v_d)^\top$ are non-integer pixel locations, the image intensity $I_d(u_d, v_d)$ must be estimated at the *non-integer* pixel location (u_d, v_d) . The most simple way to do it is through *nearest-neighbor interpolation*, i.e. approximating $I_d(u_d, v_d) \simeq I_d(\lfloor u_d \rfloor, \lfloor v_d \rfloor)$, where $\lfloor x \rfloor$ denotes the closest integer to x .

- (`undistort_image.py`) Write a function `undistort_image` that performs the image undistortion using Equation 3 and nearest-neighbor interpolation. The expected output is shown in Figure 7 (left image).

Note *for* loops are very inefficient in Matlab, although we do it here for simplicity. As an additional question, you can try to implement the `undistort_image` function using vectorization (hint: Matlab's and Python's `reshape` function might be handy). The resulting code will be faster.

Note Matlab has a `imwarp` function dedicated to this kind of operations, similar to the OpenCV function `cv2.warpAffine` in Python.

Bonus exercise Implement *bilinear interpolation* to get rid of the artifacts introduced by nearest-neighbor interpolation (see Figure 7 for comparison).

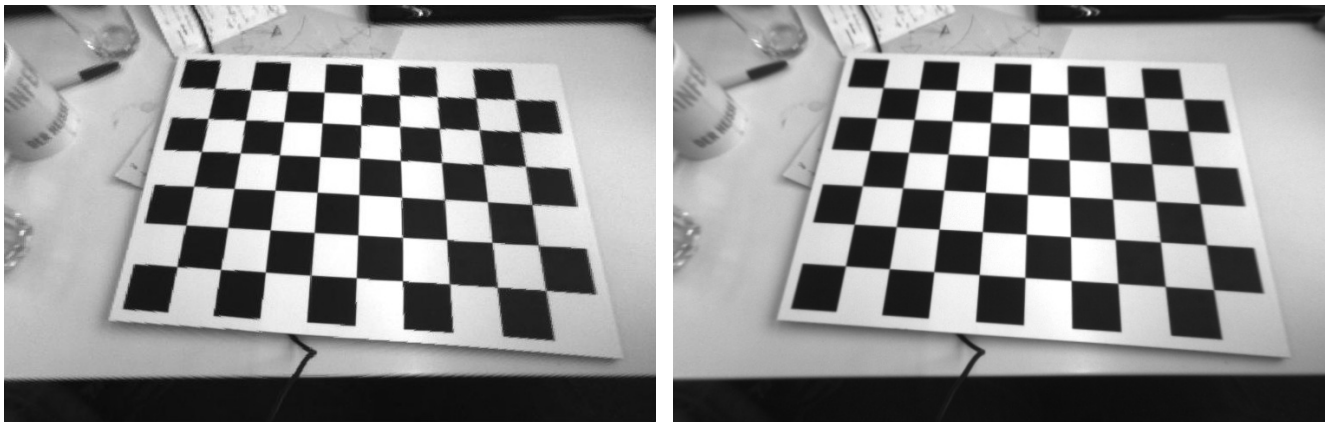


Figure 7: Undistorted images. Left: nearest-neighbor interpolation (observe the artifacts on the edges). Right: bilinear interpolation.

4 Numerical Exercises

1. Which of the following points written in homogeneous coordinates represent the same point in $\mathbb{P}^2$ as $[6, 3, 2]$?

(a) $[18, 9, 6]$ (b) $[12, -6, 4]$ (c) $[1, \frac{1}{2}, \frac{1}{3}]$ (d) $[1, 2, 3]$

2. Consider the following projective transformation $\lambda \tilde{\mathbf{q}} = H {}_W\mathbf{P}$, which maps a world point ${}_W\mathbf{P}$ to the homogeneous image point $\tilde{\mathbf{q}}$ with

$$H = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Compute the vanishing points for lines in the XZ plane in the world frame that are (1) parallel to the Z -axis, and (2) at 45° to the Z_W axis.

3. Consider a camera with the following camera matrix

$$K = \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix}.$$

State the modified camera matrix $\hat{K}$ if the image resolution is increased by a factor of two while the sensor size remains the same.

4. Compute the homogeneous image point $\tilde{\mathbf{q}}$ in pixels corresponding to the point ${}_W\mathbf{P} = [2, 5, 5]^\top$ (meters). Consider the extrinsics $\mathbf{R}_{CW}$ and ${}_C\mathbf{t}_{CW}$ (meters) and camera matrix K

$$\mathbf{R}_{CW} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad {}_C\mathbf{t}_{CW} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \quad K = \begin{bmatrix} 420 & 0 & 355 \\ 0 & 420 & 250 \\ 0 & 0 & 1 \end{bmatrix}.$$

5. Find the world-to-camera rigid-body transformation $\mathbf{T}_{CW}$ for the arrangement in Figure 8. State both $\mathbf{R}_{CW}$ and ${}_C\mathbf{t}_{CW}$.

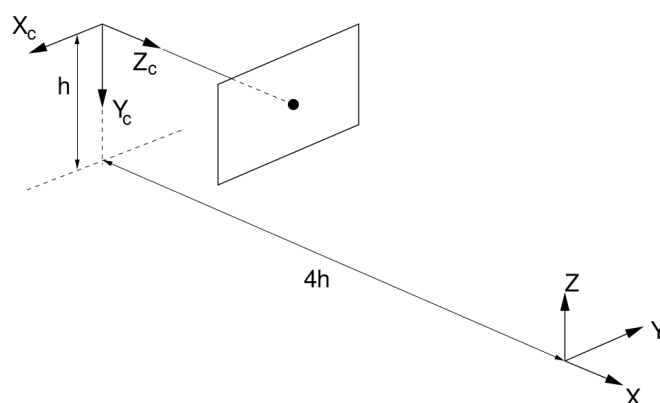


Figure 8: World frame W (axes X, Y, Z - subscript W omitted) and camera frame C (axes X_C, Y_C, Z_C). Distances are given in terms of h .

6. A camera with frame C is initially aligned with the world frame W and located at the same origin. Therefore, a world point ${}_W\tilde{\mathbf{P}}$ projects to the homogeneous image point $\tilde{\mathbf{q}}$ according to

$$\lambda \tilde{\mathbf{q}} = K \begin{bmatrix} I & \mathbf{0} \end{bmatrix} {}_W\tilde{\mathbf{P}}.$$

The camera is then rotated about its optical centre, producing a new camera frame C' . Let $\mathbf{R}_{C'C}$ map vectors from C to C' . Show that all image points, independent of depth, undergo the projective transformation

$$\tilde{\mathbf{q}}' \sim K \mathbf{R}_{C'C} K^{-1} \tilde{\mathbf{q}},$$

where $\sim$ denotes equality up to a nonzero scale.

7. A weak perspective projection comprises an orthographic projection onto the plane $Z_C = Z_A$ followed by perspective projection onto the image plane. Here, Z_A is an assumed scene depth; the subscript A denotes the approximation and is not a coordinate frame.

- (a) Derive the homogeneous relationship between a camera-frame point ${}_C\tilde{\mathbf{P}} = (X_C, Y_C, Z_C, 1)^\top$ and its image $(su_A, sv_A, s)^\top$ under weak perspective projection.
- (b) Show that the error $(u - u_A, v - v_A)$ introduced by the weak perspective approximation is given by

$$\left(\frac{\Delta Z}{Z_A} (u - u_0), \frac{\Delta Z}{Z_A} (v - v_0) \right)$$

where $\Delta Z \equiv Z_A - Z_C$.

- (c) Under what viewing conditions is weak perspective a good camera model? What are its advantages?